Shiv Nadar University Chennai

Name of Student	**KR Srinidhi Rajamane**	Reg. No. **24011101059**
Degree & Branch	B.Tech Artificial Intelligence & Data Science Section A	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	Output Range	Typical Applications
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

https://archive.ics.uci.edu/dataset/267/banknote+authentication

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

Plot	**Purpose**	**Expected Inference**
Feature Histograms	Distribution Analysis	Identify skewness and outliers
Correlation Heatmap	Feature Relationship	Observe highly correlated features
Scatter Plot	Class Distribution	Determine whether classes are approximately linearly separable
Training Error vs Epoch	Learning Behaviour	Verify convergence of the perceptron
Weight Evolution	Parameter Analysis	Observe stabilization of learned weights
Bias Evolution	Parameter Analysis	Understand the role of the bias term
Confusion Matrix	Performance Evaluation	Interpret TP, FP, TN and FN
Learning Rate Comparison	Hyperparameter Study	Compare convergence behaviour for different learning rates
Decision Boundary (Optional)	Visualization	Illustrate the separating hyperplane using two selected features

8. Performance Tables

Training Summary

Dataset Size	1372 samples
Train/Test Split	80% / 20% (1097 train / 275 test)
Learning Rate	0.01
Epochs	50 (ran for the full budget; did not reach zero error)
Final Weights	Variance: −0.1637, Skewness: −0.1767, Curtosis: −0.1895, Entropy: 0.0040
Final Bias	0.2400
Accuracy	0.9709 (97.09%)
Precision	0.9385 (93.85%)
Recall	1.0000 (100.00%)
F1-score	0.9683 (96.83%)

Epoch-wise Learning

(Weight 1 = Variance, Weight 2 = Skewness; selected epochs shown for brevity)

Epoch	Errors	Weight 1	Weight 2	Bias
1	175	−0.0744	−0.0491	0.0700
5	32	−0.0964	−0.0952	0.1200
10	36	−0.1125	−0.1215	0.1500
20	20	−0.1231	−0.1465	0.1800
30	22	−0.1389	−0.1609	0.2000
40	18	−0.1499	−0.1795	0.2200
47	16	−0.1642	−0.1847	0.2300
50	27	−0.1637	−0.1767	0.2400

9. Additional Tasks

9.1 Step vs. Sigmoid Activation

Property	**Step**	**Sigmoid**
Output	Hard $\{0, 1\}$	Smooth $(0, 1)$
Derivative	Zero everywhere, undefined at $z = 0$	Well-defined, non-zero almost everywhere
Gradient flow	None – blocks backpropagation	Enables gradient-based learning
Confidence info	None (hard decision only)	Yes, output is a probability-like score

The Step function is not used in modern deep learning because it is non-differentiable, so gradient descent and backpropagation cannot flow a useful error signal back through it. A single perceptron only ever needs a hard yes/no decision, but a multi-layer network needs every layer's error to be propagated backward using the chain rule, which requires a smooth, differentiable activation such as Sigmoid, Tanh or ReLU.

9.2 Comparison with Scikit-learn's Perceptron

The from scratch implementation matched the Scikit-learn's accuracy, precision, recall and F1 score.

9.3 Effect of Learning Rate

This was repeated for various learning rates, each trained for 50 epochs; the resulting convergence curves are plotted and the final error counts are tabulated. In short the larger step sizes overshoot the decision boundary on data that is not perfectly linearly separable instead of settling into it.

9.4 Why a Single Layer Perceptron Cannot Solve XOR

XOR is not linearly separable. Since a single-layer perceptron can only represent one linear decision boundary, it is mathematically incapable of solving XOR. Solving XOR requires at least one hidden layer with a non-linear activation function which remaps the inputs into a new feature space in which the classes become linearly separable.

9.5 Effect of Feature Normalization on Convergence

Before scaling Skewness has a much larger numeric range than Entropy, so the weighted sum would be dominated by whichever feature happens to have the largest raw magnitude, not by which feature is actually most informative for classification. This slows and destabilizes convergence.

10. Source Code

The complete implementation was carried out in Python using NumPy, Pandas, Matplotlib, Seaborn and Scikit-learn. The full, executable source code (organized task-wise, exactly as it was run) is available on GitHub:

https://github.com/Srinidhi-Krishna/Deep-Learning-Lab/blob/main/Lab_1/DL_LAB_1.ipynb

11. Experimental Results

11.1 Dataset Exploration (Task 1)

The loaded data set is fine. It has 1372 rows and 5 columns (4 features + class label). Skewness has the widest spread, while Entropy is the most tightly packed.

11.2 Data Preprocessing (Task 3)

After `MinMaxScaler`, all four features got squeezed into $[0, 1]$.

11.3 Model Training (Task 5)

Trained for a fixed 50 epochs. Though initial epochs started with considerable errors, it dropped fast and then just kept bouncing between for the rest of training instead of settling down.

11.4 Model Evaluation (Task 6)

Final-epoch weights on the test set gave:

Metric	Value
Accuracy	0.9709 (97.09%)
Precision	0.9385 (93.85%)
Recall	1.0000 (100.00%)
F1-score	0.9683 (96.83%)

Confusion matrix on the 275 test samples:

$$\begin{bmatrix} 145 & 8 \\ 0 & 122 \end{bmatrix}$$

145 authentic notes were correctly caught, 8 authentic notes were wrongly flagged as forged and zero forged notes slipped through as authentic. Hence the recall on the forged class is a perfect 1.0.

11.5 Effect of Learning Rate (Task 7)

Tried three rates, each trained fresh for 50 epochs:

Learning Rate (η)	**Epochs Run**	**Final Misclassified**
0.001	50	16
0.01	50	27
0.1	50	27

I expected the smallest rate to be the slowest one, but surprisingly it ended up with the fewest errors. Small steps mean that the weight vector moves cautiously and does not overshoot.

11.6 Comparison with Scikit-learn (Task 9, Optional)

Ran scikit-learn's `Perceptron` with the same settings on the same split and got identical numbers to the from-scratch version for accuracy, precision, recall and F1 score.

12. Generated Plots with Interpretation

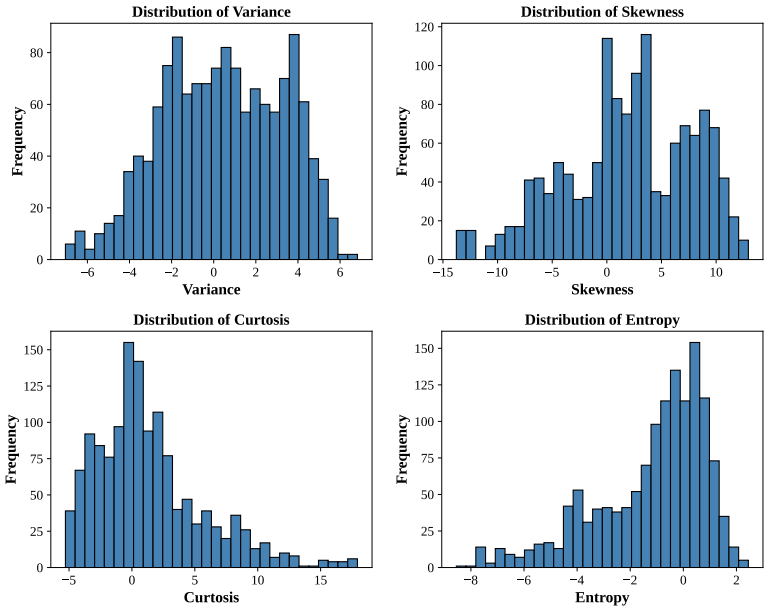

Figure 1: Frequency distribution histograms of Variance, Skewness, Curtosis and Entropy.

Interpretation: None of the four features appears bell-shaped. The raw features are not normally distributed. This is not a problem for a perceptron, but this is why I decided to go with min-max scaling instead of standardization.

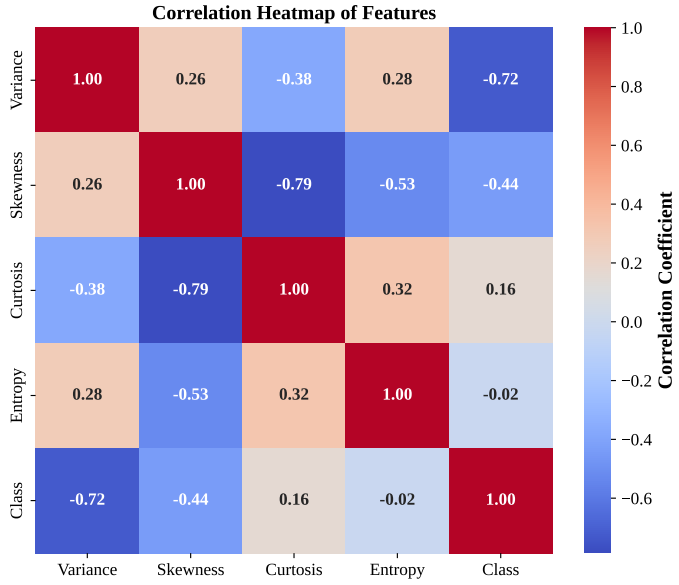

Figure 2: Pearson correlation heatmap among the four features and the class label.

Interpretation: Of the four, Variance has the strongest correlation to Class, which is consistent with the

final weights.

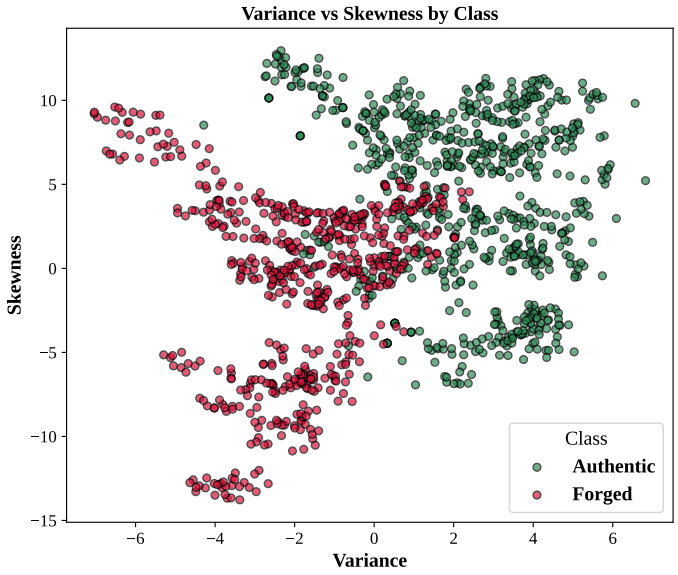

Figure 3: Scatter plot of Variance versus Skewness, colour-coded by class.

Interpretation: This plot is what convinced me that a linear classifier could actually work here as authentic and forged points sit mostly in different regions.

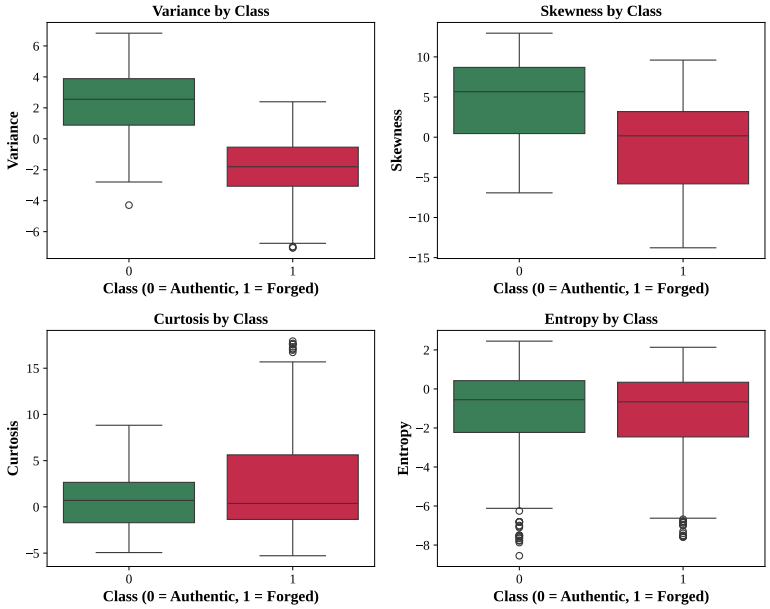

Figure 4: Boxplots of each feature grouped by class.

Interpretation: Same story as the scatter plot with median Variance and Skewness for authentic notes sit above forged notes.

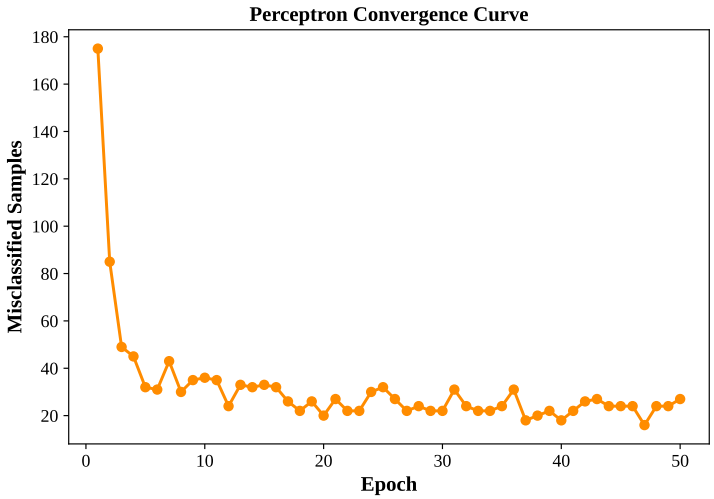

Figure 5: Misclassified training samples per epoch (learning rate 0.01).

Interpretation: Errors drop fast at first and then flatten into a noisy plateau for the rest of the training. It never hits zero, which means that the four features together aren't perfectly linearly separable.

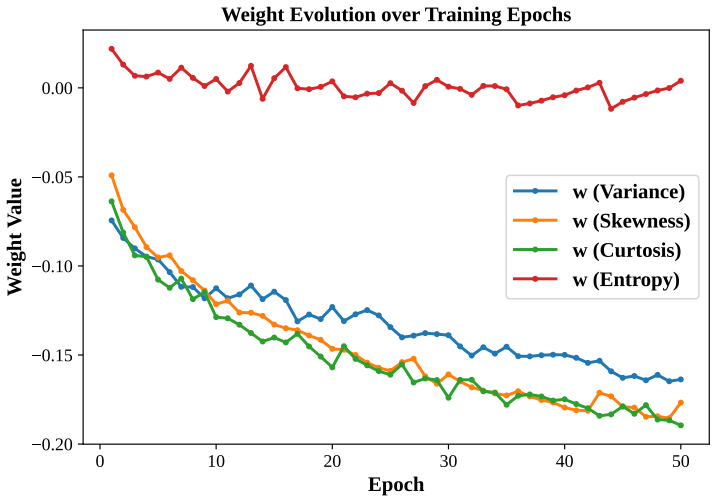

Figure 6: Evolution of the four feature weights across training epochs.

Interpretation: All four weights move fast early on and then level off into small oscillations instead of a flat line, matching the fact that error never hits zero.

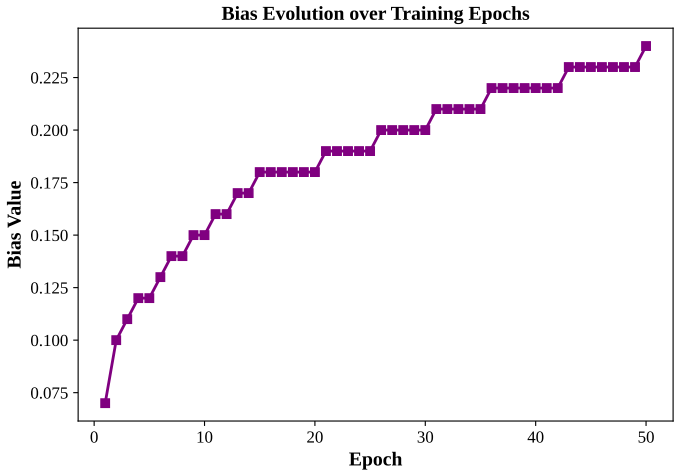

Figure 7: Evolution of the bias term across training epochs.

Interpretation: The bias climbs in a rough staircase pattern from 0 up to about 0.24 and it's still climbing at epoch 50, which means the model is still making corrections right up to the end.

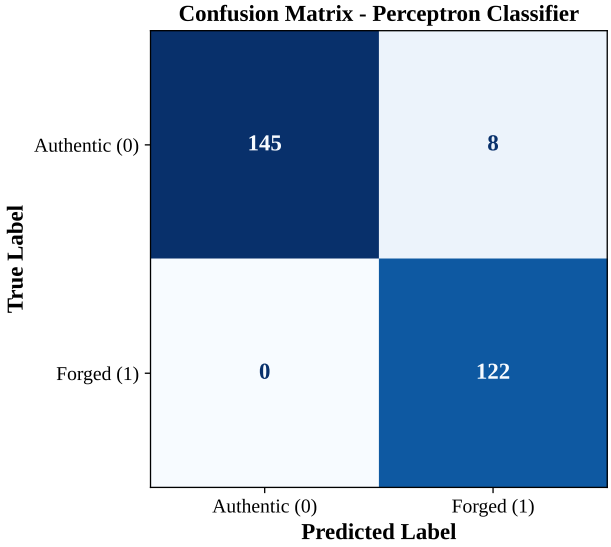

Figure 8: Confusion matrix on the held-out test set (275 samples).

Interpretation: Only 8 out of 275 test notes were misclassified, and all 8 were authentic notes flagged as forged notes but none of the forged notes got through as authentic. For currency verification that is a good error pattern to have, since a false alarm on a genuine note is a lot less costly than letting a fake one pass.

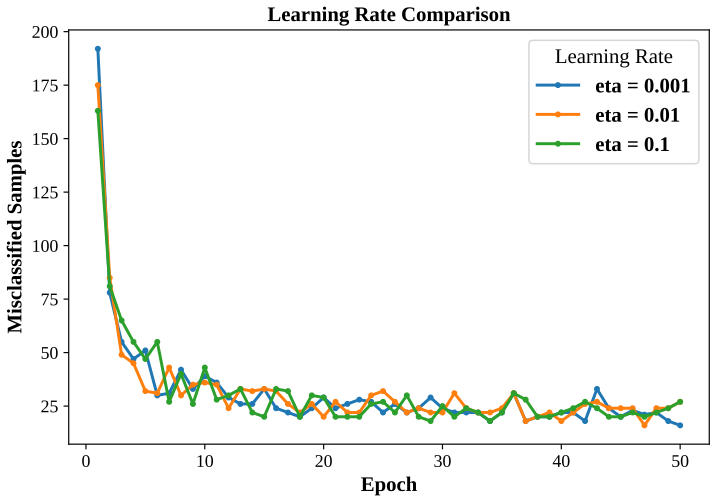

Figure 9: Convergence behaviour of the perceptron under three different learning rates.

Interpretation: All three curves drop rapidly at first but settle at different noise floors. Smaller steps here land closer to a good boundary, larger steps keep the weight vector jumping around more.

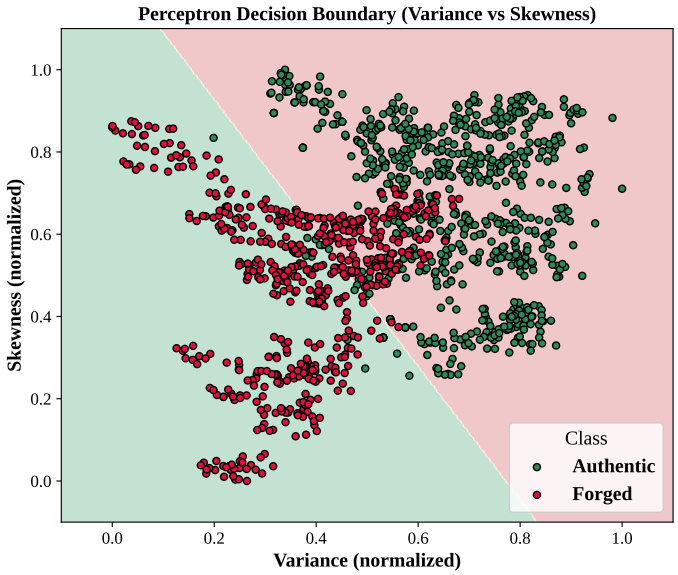

Figure 10: Decision boundary learned by a 2-feature perceptron (Variance vs Skewness).

Interpretation: With just two features, the boundary is seen having the most authentic points on one side, most forged on the other, with a few points from each class right on or just past the line.

13. Discussion

The from-scratch perceptron does well here with a test accuracy of 97.09% and 100% recall on the forged class. A fake note never passed through as genuine on the test split.

The learning-rate points out that the smallest rate ($\eta = 0.001$) ended up with fewer errors than the larger ones, which goes against the usual "bigger step = faster learning" intuition.

Normalization mattered too as Skewness has a much bigger raw scale than Entropy, so without scaling, Skewness would dominate the weighted sum.

14. Conclusion

A Single Layer Perceptron is implemented from scratch and it is tested on the UCI Banknote Authentication dataset. This resulted in 97.09% accuracy with 93.85% precision, 100% recall and 96.83% F1 on the test set, and the output matched scikit-learn's built-in `Perceptron` exactly under the same hyperparameters.

The training curves showed that the four features are only approximately linearly separable, so the model settles into a noisy low-error region instead of reaching zero, and the learning-rate experiment showed that a smaller step size can actually converge better than a larger one on this kind of data.

15. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.